

CS-2001 – Data Structures – Fall 2026

ASSIGNMENT # 01

Maximum Marks: 100

Post Date: Sep 10, 2026

Due Date: Sep 19, 2026

Submission: Google Classroom

IMPORTANT INSTRUCTIONS

- Plagiarism of any kind will not be tolerated.
- You need to submit/ upload the SCAN copy of your hand-written assignment on google classroom.
- Late submissions will not be accepted.

Question 01:

Fiber-optic company is installing cables at different locations and has recorded the required cable lengths as 85, 42, 120, 35, 67, 50. The installation team needs these lengths arranged in **ascending order** before preparing the final installation plan. The process starts by considering the first value as arranged, then takes the next value and stores it temporarily. Compare this value with the previously arranged values, **shift larger values one position to the right**, and place the stored value into its correct position. Your program must take input from the user, display the original array, display the array after each iteration, and display the final sorted array. **Do not use sort (), another array, or repeated swapping; use a temporary variable and shifting as described.**

Your task is to identify the sorting algorithm represented by this scenario and then implement the identified algorithm in C++.

Question 02:

A library has a **dynamically allocated array of shelves**, where each shelf stores the number of books it can hold. The librarian wants to keep the shelf capacities arranged in **ascending order** for efficient organization. During renovation, a **new shelf may need to be inserted in the middle of the existing shelves**, so the program must create a larger dynamic array, copy the existing shelf capacities, and place the new shelf at the specified position. After insertion, the shelf capacities must again be arranged in ascending order using the given sorting logic.

Do not use sort() or STL containers; use dynamic memory (new/delete), handle shelf insertion at any position, and implement the required sorting logic manually. Your task is to identify the

sorting algorithm represented by the sorting process and implement the complete solution in C++.

Input:

Enter number of shelves: 5
Enter book capacity of each shelf:
100 60 150 80 120
Enter position to add the new shelf (0 to 5): 2
Enter capacity of the new shelf: 70

Output:

Original shelf capacities: 100 60 150 80 120
Shelves after adding new shelf: 100 60 70 150 80 120
Shelves after sorting: 60 70 80 100 120 150

Question 3:

A warehouse management system stores the package weights in the following order: **120, 35, 90, 15, 75, 10, 60, 25**. The system needs to arrange these weights in **ascending order**. However, instead of repeatedly comparing only neighboring elements, the sorting process is designed to first compare elements that are separated by a **large distance**. This distance is called the **gap**. Initially, the gap is related to the size of the array. After every pass, the gap is reduced using a **shrink factor of 1.3**. Elements separated by the current gap are compared and swapped when they are in the wrong order. The process continues until the gap becomes **1** and a complete pass is performed without any swaps.

Your program must take the array values as input from the user, display the original array, display the gap value used in each iteration/pass, display the array after each pass, display the final sorted array, use a gap-based comparison approach, and must not use sort(), another array, or any other sorting algorithm.

Question 04:

A KFC branch manages its delivery riders in a **circular way**, where each rider is connected to the next rider, and the last rider points back to the first rider. This allows delivery orders to be assigned to riders continuously in a circular manner.

Each rider node should contain **Rider ID, Rider Name, and Number of Assigned Orders**. The system should support **insertion at the beginning, insertion at the end, insertion at a specific position, deletion from the beginning, deletion from the end, deletion from a specific**

position, searching for a rider, updating rider information, displaying all riders, counting the total riders, and traversing the riders starting from a selected rider.

The implementation must correctly maintain the circular connection after every insertion and deletion, handle an empty list and invalid positions, and **must not use STL containers such as list or vector.**

Question 05:

A beauty company operates two branches, **GlowCare** and **BeautyHub**, and each branch maintains its own Singly Linked List of beauty products. Each product node contains a **Product ID, Product Name, Category, Price, and pointer to the next node.** The products in each branch are maintained in **ascending order of Product ID.**

At the end of the month, the company wants to combine the product records of both branches into a single master product list. Write a C++ program that dynamically creates the two Singly Linked Lists and allows the user to add products to each list while maintaining the ascending order of Product ID. The program must traverse and display both branch lists before merging, then merge the two lists into a single sorted master list by correctly adjusting the existing next pointers. The merged list must not contain duplicate Product IDs; if the same Product ID exists in both lists, retain the product with the **lower price** and discard the other node. Finally, traverse and display the complete master list and display the total number of products and the total inventory value of the remaining products. The program must perform the operations using linked-list pointers only and must not use arrays, STL lists, sort(), or any built-in merge function. ***Your task is to design and implement the required Singly Linked List operations in C++.***

Question 06:

You are building a sophisticated music playlist manager using a singly linked list where each node contains a **Song object.** Your task is to implement a complete C++ program that can store songs in a linked list and perform all fundamental operations—insertion at any position, deletion at any position, and searching.

The **Song** class contains the song's title, **genre** (pop, rock, jazz, classical, or electronic), **duration** in seconds, and a special Boolean flag indicating if the song contains **explicit content.**

Explicit songs are marked with the flag set to true. Explicit songs cannot be deleted directly. They first require not to be explicit, then should be deleted.

Songs of the **same genre** cannot be placed next to each other in the playlist. When you try to insert a song at a position where it would be adjacent to another song of the same genre, your

implementation must automatically find the next valid position where this genre conflict doesn't occur.

The total duration of any three consecutive songs in the playlist cannot exceed 10 minutes (600 seconds). When adding a new song, if placing it at the desired position would violate this duration limit with its neighbors, your code must find an alternative position that maintains balanced listening sessions.

Every time you perform 3 operations on the playlist (any combination of insert, delete, or search), above constraints must be satisfied.

Question 07

You are required to implement a **Turn-Based Battle Simulator game** using Doubly Linked Lists where each node contains a **Combatant object** with properties like **name, health points (1-70), both inclusive and attack power (1 to 5), both inclusive**. This game simulates a classic RPG-style battle between a player's party of heroes and a team of enemies. The core challenge involves managing two teams - **one for the hero's team and another for the enemy team**.

The hero's team should be initialized with five heroes. The enemy team should contain five enemies in this sequence. The battle follows a structured turn based sequence where the player hero always attacks first in each round, followed by the enemy counterattack.

Any member of a team can attack any opponent. During each turn, the program will first ask the player to **choose an attacker (by name or position)** and then select a target from the opponent's team (also by name or position). The attacker and target must exist in their respective teams. Once attacker is chosen, the attacker will strike on opponent and the opponent's health will be reduced by:

Damage = Attack Power × Random Value (where the random value is between 1 and 3, inclusive).

If any combatant's health reaches zero or below, they are immediately removed from their team. After every turn, only the names of the remaining team members from both sides will be displayed. **(do not display health points and attack power of combatant while playing the game).**

The winning conditions are based on both survival and strength. If one team is completely eliminated, the other team wins instantly. However, if the combat lasts until a fixed number of rounds (10 Minimum), the winner is decided by comparing the total health points of the remaining members. If both teams have the same total health, the match ends in a draw. At the end of the game, display the names, health points, and attack power of the surviving members of the winning

team, or in the case of a draw, display the remaining members of both teams, with remaining health points and attack power.

Question 08

You are developing a game called **Undo Battle** in which a player moves along a one-dimensional path starting from position 0. The player can perform four types of commands: **R** moves the player one position to the right, **L** moves the player one position to the left, **J** moves the player two positions to the right, and **B** performs an **Undo** operation. Every valid movement (R, L, or J) must be stored in a stack so that the most recent movement can be undone. When the player performs **B**, the most recent valid movement must be removed from the stack and the player must return to the position before that movement. An Undo operation itself must not be stored in the stack. A movement that would take the player to a position less than 0 is considered invalid and must not be stored in the stack. If the player requests an Undo when the stack is empty, nothing should happen. You are required to implement the stack using a **linked list** and process n commands given as input. At the end, display the player's final position, the total number of successful movements, the total number of successful Undo operations, and the remaining elements in the stack. **Do not use STL stack or any other built-in stack data structure.**

Question 09

You are developing an **Emergency Room Game** in which patients arrive at a hospital and wait to be treated. Each patient has a unique **Patient ID** and a **severity level**, where 1 represents **Critical**, 2 represents **Serious**, and 3 represents **Normal**. Patients normally arrive and wait in the order in which they enter the hospital; however, when the doctor is ready to treat a patient, a Critical patient must always be treated before a Serious or Normal patient, and a Serious patient must always be treated before a Normal patient. If two or more patients have the same severity level, the patient who arrived first must be treated first.

The input consists of a sequence of **ARRIVE ID SEVERITY** and **TREAT** operations. When an **ARRIVE** operation is encountered, the patient must be added to the appropriate queue, and when a **TREAT** operation is encountered, the patient with the highest severity must be removed according to the above rules. You are **not allowed to use a priority queue**; instead, implement the solution using normal queues implemented with an **array**. You may use multiple queues to maintain the required priority and FIFO order.

At the end, display the treatment order, the total number of patients treated, the total number of patients remaining, and the number of Critical, Serious, and Normal patients still waiting. **Do not use STL queue, priority_queue, vector, deque, or any other built-in data structure.**

Question 10

You are developing a game called **Treasure Battle** in which several players are waiting to enter a treasure cave. Each player has a unique ID and starts with `100` energy points. Players enter the cave in the order in which they arrive. During the game, players take turns one by one. On each turn, the player at the front gets exactly one command to perform. After completing the command, if the player is still active, they go to the end of the waiting line and the next player gets a turn. If a player's energy becomes `0` or less, that player is eliminated and does not return to the waiting line.

The available commands are `F` (move forward), `B` (move backward), `T` (collect treasure), and `U` (undo the previous valid action). A forward move decreases the player's energy by `10`, moving backward has no effect on energy, and collecting treasure increases the player's energy by `20`. Whenever a player performs `F`, `B`, or `T`, the action must be remembered because the player may later use `U`. When `U` is performed, only the **most recent action of that player** is cancelled and its effect must be reversed. An `U` command itself cannot be cancelled. If the player has no previous action to undo, the command has no effect. Each player must maintain their own history of actions independently.

The game is given as a sequence of commands. You must process all commands according to the rules above. At the end, display the order in which players were eliminated, followed by the IDs and remaining energy of all active players in the waiting line. Also display the number of actions that can still be undone for each remaining player. You must implement the required data structures yourself using arrays. **Do not use STL containers such as `stack`, `queue`, `vector`, or `deque`.**

THE END!